PROJETO DE FACULDADE · ANDROID · KOTLIN

Carona Universitária

Guia de instalação, configuração e explicação de cada funcionalidade do código

O que é: aplicativo Android nativo (Kotlin + Jetpack Compose) para alunos, professores e colaboradores de uma faculdade organizarem caronas compartilhadas, com acesso restrito a e-mail institucional.

Este guia ensina: (1) o que instalar no PC e como configurar; (2) como rodar o app e a API; (3) como o código funciona, funcionalidade por funcionalidade; (4) um roteiro de apresentação e as perguntas técnicas mais prováveis.

Sumário

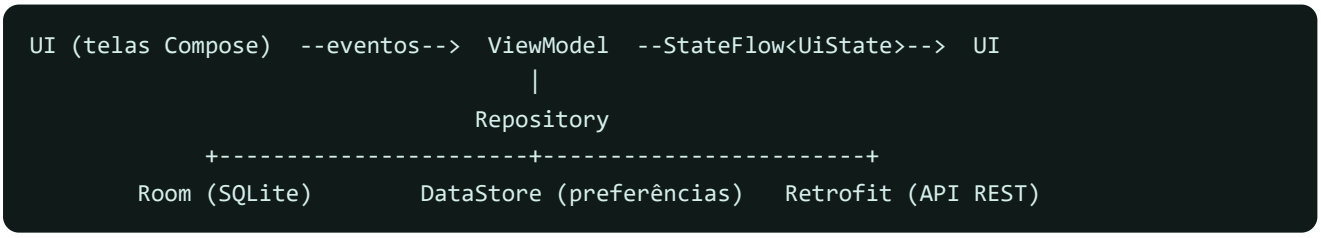
1. Visão geral e arquitetura em uma página
2. O que instalar no computador
3. Configuração passo a passo
4. Problemas comuns e soluções
5. Estrutura de pastas do projeto
6. Como o código é organizado (camadas, estado, injeção de dependências)
7. Explicação de cada funcionalidade
8. Banco de dados, DataStore, API e sincronização
9. Navegação, diálogos de confirmação e visual
10. Testes unitários
11. Roteiro de apresentação (8 minutos)
12. Perguntas técnicas prováveis, com respostas
13. Glossário rápido

1. Visão geral e arquitetura em uma página

Problema: estudantes gastam muito ou perdem tempo para chegar ao campus, enquanto carros circulam com vagas vazias. **Solução:** uma comunidade fechada em que motoristas oferecem caronas e passageiros pedem vaga, conversam por chat e confirmam o trajeto.

Duas partes:

- **frontend/** — o aplicativo Android (Kotlin + Jetpack Compose). Tudo da interface é escrito em Kotlin; não há HTML/CSS.
- **backend/** — uma API REST de desenvolvimento (Node.js + json-server) que guarda as caronas em `db.json`.



Regra de ouro: **a tela só mostra o estado e avisa eventos; a ViewModel decide; o Repository fala com banco/rede.** Nenhuma ViewModel recebe `Context`.

2. O que instalar no computador

Programa	Para que serve	Onde baixar	Precisa?

Android Studio (versão recente, ex.: Ladybug ou mais nova)	Editor oficial do Android. Já traz o JDK (JBR 17/21), o Gradle gerenciado, o gerenciador de SDK e o emulador.	developer.android.com/studio	**Sim**
Android SDK Platform 35 (Android 15) + Build-Tools + Platform-Tools + Emulator	Bibliotecas e ferramentas para compilar e rodar o app (o projeto usa `compileSdk 35`). Instalados de dentro do Android Studio.	Android Studio → Settings → Languages & Frameworks → Android SDK	**Sim**
Emulador Android (AVD) ou celular Android 8.0+	Onde o app roda (minSdk 26 = Android 8.0).	Android Studio → Device Manager	**Sim** (um dos dois)
Node.js LTS (inclui o npm)	Executa o backend de desenvolvimento (json-server).	nodejs.org	**Sim**, para a lista de caronas vir da API
Git	Controle de versão e envio do código ao GitHub.	git-scm.com	Sim (entrega)
VS Code + extensão **Prettier**	Opcional: editar Markdown/JSON do repositório e formatá-los.	code.visualstudio.com	Opcional

Requisitos do computador: Windows 10/11 de 64 bits, 8 GB de RAM (16 GB é bem melhor para o emulador), cerca de 10 GB livres em disco e **virtualização ativada** na BIOS (Intel VT-x / AMD-V) para o emulador ficar rápido. Na primeira abertura é preciso internet: o Gradle baixa o Gradle 8.9 e as bibliotecas do projeto (alguns minutos).

JDK: não instale o Java separado. Use o que vem no Android Studio (JBR). O projeto compila com Java 17.

3. Configuração passo a passo

3.1 Instalar e preparar o Android Studio

1. Instale o Android Studio com as opções padrão (aceite instalar o Android SDK e o Android Virtual Device).
2. Abra **More Actions → SDK Manager** (ou **Settings → Languages & Frameworks → Android SDK**).
3. Na aba **SDK Platforms**, marque **Android 15.0 (API 35)**. Na aba **SDK Tools**, confirme: Android SDK Build-Tools, Android SDK Platform-Tools, Android Emulator. Clique em **Apply**.

3.2 Criar o emulador

1. **Device Manager → Create Device**.
2. Escolha um modelo (ex.: *Pixel 8*) e uma imagem do sistema **API 35** com *Google APIs* ou *Google Play* (necessário para o botão "Ver rota no mapa" abrir o Google Maps). Baixe a imagem e conclua.

3.3 Abrir o projeto

1. **File** → **Open** e selecione a pasta `frontend` (e não a pasta raiz do repositório).
2. Aguarde o **Gradle Sync** terminar (barra de progresso no canto inferior). Se aparecer um aviso para instalar uma plataforma ou ferramenta, clique no link para instalar.
3. Se o Studio perguntar o *Gradle JDK*, escolha o **jbr** embutido (File → Settings → Build Tools → Gradle → Gradle JDK).
4. O arquivo `frontend/local.properties` guarda o caminho do SDK (`sdk.dir=...`). O Studio o recria sozinho se o caminho for diferente no seu PC; ele não vai para o GitHub.

3.4 Subir o backend (API)

```
cd backend
npm install
npm start
```

- Teste no navegador do PC: `http://localhost:3000/rides` deve mostrar uma lista JSON com 3 caronas de exemplo.
- Deixe esse terminal aberto enquanto usa o app.
- O emulador enxerga o seu PC pelo endereço especial `http://10.0.2.2:3000/` . Esse é o valor padrão do projeto (`apiBaseUrl` em `frontend/gradle.properties`).

3.5 Rodar o app

1. Na barra superior, escolha o emulador criado e a configuração **app**.
2. Clique em **Run** ▶. O app abre na tela de login.
3. Cadastre-se com um e-mail `...@algumacoisa.edu.br` ou `...@habil.com.br` (outros domínios são recusados de propósito).

3.6 Rodar os testes

No Android Studio: clique com o botão direito em `app/src/test` → **Run Tests**. Pelo terminal, dentro de `frontend` , use `gradlew testDebugUnitTest` (o Studio fornece o Gradle; se o arquivo `gradlew` não existir, gere com *File → Sync* ou use a aba Gradle do Studio: *Tasks → verification → testDebugUnitTest*).

3.7 Usar um celular físico em vez do emulador

1. No celular: ative **Opções do desenvolvedor** (toque 7 vezes em "Número da versão") e **Depuração USB**; conecte por cabo e autorize o computador.
2. O celular e o PC precisam estar na **mesma rede Wi-Fi**. Descubra o IP do PC (`ipconfig` , ex.: 192.168.0.15).
3. Altere `apiBaseUrl` em `frontend/gradle.properties` para `http://192.168.0.15:3000/` .
4. Em `app/src/main/res/xml/network_security_config.xml` , acrescente uma linha `<domain includeSubdomains="false">192.168.0.15</domain>` (o app só permite HTTP sem criptografia para endereços listados).
5. Libere a porta 3000 no Firewall do Windows se o celular não conseguir acessar.

4. Problemas comuns e soluções

Sintoma	Causa provável e solução
"Não foi possível carregar" na lista de caronas	O backend não está rodando ou o endereço está errado. Rode `npm start` em `backend` e confirme `http://localhost:3000/rides`. No emulador use 10.0.2.2.
Gradle Sync falha / "SDK location not found"	Instale o SDK Platform 35 no SDK Manager e confira o `sdk.dir` em `frontend/local.properties`.
Erro de versão do Java / JDK	Em Settings → Gradle, use o **jbr** do Android Studio (17 ou 21).
Sync muito lento ou sem internet	Primeiro sync baixa centenas de MB. Aguarde e verifique proxy/antivírus.
Emulador muito lento ou não inicia	Ative a virtualização na BIOS e o recurso "Plataforma de Hypervisor do Windows". Feche programas pesados.
App fecha ao abrir depois de atualizar o código	O banco local foi recriado. Desinstale o app do emulador (ou "Wipe Data" no Device Manager) e rode de novo.
"Ver rota no mapa" mostra que não há app de mapas	Use uma imagem de emulador com Google APIs/Play, ou abra o Maps no aparelho.
Porta 3000 ocupada	Feche o outro processo ou rode o json-server em outra porta e ajuste `apiBaseUrl`.
Cadastro recusa meu e-mail	Só são aceitos domínios `edu.br` (e subdomínios) e `habil.com.br`. Edite `EmailPolicy.allowedDomains` para incluir o da sua faculdade.

5. Estrutura de pastas do projeto

```
projeto-transporte/
├── frontend/                      app Android (abrir esta pasta no Android Studio)
│   ├── build.gradle.kts, settings.gradle.kts, gradle.properties
│   ├── gradle/libs.versions.toml  versões de todas as bibliotecas
│   └── app/
│       ├── build.gradle.kts       configuração do módulo e dependências
│       └── src/
│           ├── main/
│           │   ├── AndroidManifest.xml
│           │   ├── res/           strings, cores, tema, ícone, config de rede
│           │   └── java/br/com/habil/carona/
│           │       ├── domain/    modelos e regras puras (sem Android)
│           │       ├── data/      local (Room, DataStore), remote (Retrofit),
│           │       │            mapper, repository
│           │       └── ui/        theme, components, navigation e um pacote por módulo
│           └── test/              testes unitários
├── backend/                       API de desenvolvimento (db.json, package.json)
├── docs/                          documentação em Markdown
└── README.md, CONTRIBUTING.md, .prettierrc.json, .gitignore
```

6. Como o código é organizado

6.1 As camadas

Camada	Pasta	O que faz
Domínio	`domain/`	Modelos (`Ride` , `User` , `Contact` ...) e regras que não dependem do Android: validação de e-mail, de senha, do formulário e o filtro de caronas.
Data Source	`data/local` , `data/remote`	Room (tabelas e DAOs), DataStore (preferências) e Retrofit (API).
Repository	`data/repository`	Regras de negócio. Junta banco e rede, valida permissões e devolve `Result` (sucesso ou erro com mensagem).
ViewModel	`ui/<módulo>`	Guarda o `UiState` da tela num `StateFlow` , recebe eventos e controla os diálogos.
UI	`ui/<módulo>`	Telas em Compose. Uma função `...Route` coleta o estado da ViewModel e chama a tela `...Screen` , que só desenha.

6.2 Fluxo de dados unidirecional (UDF)

1. O usuário toca em algo → a tela chama uma função da ViewModel (evento).
2. A ViewModel chama o Repository (em corrotina, no `viewModelScope`).
3. O Repository altera o banco/rede, no `Dispatchers.IO` .

4. O Room emite o novo dado por `Flow` → a ViewModel recalcula o `UiState` → a tela recompõe sozinha.

6.3 UiState com sealed interface

Cada tela tem estados como `Loading`, `Error` e `Success` (ou `Content`). Isso obriga a tela a tratar todos os casos. Mensagens de uso único (Snackbar, voltar) saem por `SharedFlow<UiEvent>` (`ui/common/UiEvent.kt`).

6.4 Injeção de dependências manual

- `AppContainer` cria banco, DataStore, Retrofit e todos os repositórios (único lugar que conhece `Context`).
- `ViewModelFactories` monta cada ViewModel com os repositórios que ele precisa; argumentos de navegação chegam pelo `SavedStateHandle`.
- `CaronaApplication` cria o container ao iniciar o app.

7. Explicação de cada funcionalidade

7.1 Cadastro e login com e-mail institucional

O que faz: só entra no app quem tem e-mail acadêmico. No cadastro a pessoa informa nome, e-mail, se é *Estudante* (com curso), *Professor(a)* ou *Colaborador(a)*, e senha (mínimo 6 caracteres).

No código:

- `domain/validation/EmailPolicy.kt` — aceita domínios `edu.br` (e subdomínios como `aluno.faculdade.edu.br`) e `habil.com.br`; recusa Gmail etc.
- `domain/validation/PasswordPolicy.kt` — regra do tamanho mínimo da senha.
- `ui/auth/AuthViewModels.kt` — `LoginViewModel` e `RegisterViewModel`: validam os campos, mostram erros ao lado de cada campo e controlam o "carregando".
- `ui/auth/AuthScreens.kt` — telas de login e cadastro.
- `data/repository/AuthRepository.kt` — `register` e `login`. A senha nunca é guardada pura: gera-se um *salt* aleatório e guarda-se o **SHA-256(salt + senha)** (`PasswordHasher`). Ao entrar, o e-mail da sessão é salvo no DataStore.

Tabela: `users` (e-mail, nome, salt, hash, vínculo, curso, foto).

7.2 Sessão e escolha do grafo de telas

O que faz: se há sessão, o app mostra a área principal; se não, o login. Ao sair da conta, volta ao login sozinho.

No código: `ui/AppViewModel.kt` combina `currentUser` (sessão) com o tema. `MainActivity.kt` decide: `Loading` → tela de carregamento; `LoggedOut` → `AuthNavHost`; `LoggedIn` → `MainScaffold`. Não há navegação manual de login/logout: basta a sessão mudar.

7.3 Lista de caronas, busca e filtros

O que faz: mostra as próximas caronas (vindas da API e criadas no app), ordenadas por horário. Há busca por texto (bairro, cidade, rota, motorista) e filtros "Com vagas", "Meus contatos" e período (qualquer dia, hoje, próximos 7 dias). O último filtro escolhido é lembrado.

No código:

- `ui/rides/RideListViewModel.kt` — combina 5 fluxos (caronas do Room, filtros do DataStore, texto da busca, estado da sincronização, identidade/contatos) num único `RidesUiState`: **Loading** (primeira carga sem cache), **Error** (sem cache e sem rede, com botão "Tentar novamente") e **Success**. Com cache e falha de rede, aparece um Snackbar com "Tentar novamente".
- `domain/model/RideFilters.kt` — função de extensão `applyFilter` (pura e testada).
- `ui/rides/RideListScreen.kt` — campo de busca, `FilterChip`s, `LazyColumn` de `RideCard`, estado vazio ("Nenhuma carona disponível") e botão flutuante "Oferecer carona".

- O campo de busca usa um `StateFlow` separado (`searchQuery`) para o cursor não "pular" ao digitar.

7.4 Oferecer e editar carona

O que faz: formulário com saída, destino, rota aproximada (opcional), data e horário, vagas (1 a 8), veículo e observações. Ao editar, não deixa reduzir as vagas abaixo do número de passageiros já confirmados.

No código:

- `ui/rides/RideFormViewModel.kt` — guarda os campos (`RideFormFields`), compara com o estado inicial para saber se há **alterações não salvas** (`isDirty`) e valida com `validateRideForm` (campos obrigatórios, horário no futuro, vagas 1–8).
- `ui/rides/RideFormScreen.kt` — campos de texto com mensagens de erro, seletor de data (`DatePickerDialog`) e hora (`TimePicker`) em dois passos. Um `BackHandler` intercepta o botão voltar quando há alterações e abre o **AlertDialog "Descartar alterações?"**.
- `RideRepository.saveRide` — cria um UUID, grava no Room com `synced = false` e tenta enviar para a API em segundo plano.

7.5 Detalhe da carona, pedido e confirmação de vaga

Passageiro: vê os dados completos (inclusive curso/"Professor(a)" do motorista), toca em **Solicitar vaga**, acompanha o status (Pendente, Confirmada, Recusada, Cancelada), conversa no chat e pode **cancelar**. Também pode adicionar o motorista aos contatos.

Motorista: vê a lista de solicitações, com o vínculo de cada passageiro, e pode **confirmar** ou **recusar**; pode editar ou excluir a carona.

No código:

- `ui/rides/RideDetailViewModel.kt` — combina carona, quem é o usuário, solicitações, diálogo aberto e "ocupado". Define se a pessoa é motorista (`isDriver`) e qual foi o último pedido dela (`myRequest`).
- `data/repository/RequestRepository.kt` — `requestSeat` (não pode pedir na própria carona, nem em carona que já partiu, nem duplicado, nem sem vaga), `confirm` (verifica o limite de vagas), `reject` e `cancel`. Tudo dentro de `database.withTransaction`, para dois cliques simultâneos não estourarem as vagas.
- Vagas livres = vagas totais − solicitações com status CONFIRMED (calculado no SQL em `RideDao`, sem coluna duplicada).

Tabela: `ride_requests` (carona, passageiro, status, data).

7.6 Ver a rota no Google Maps

O que faz: o botão "Ver rota no mapa" abre o app de mapas com o trajeto de carro entre saída e destino.

No código: `ui/util/Intents.kt` monta a URL `https://www.google.com/maps/dir/?api=1&origin=...&destination=...&travelmode=driving` e dispara um `Intent.ACTION_VIEW`. Não precisa de chave de API. Se não houver app de mapas, mostra uma mensagem em vez de travar.

7.7 Chat motorista ↔ passageiro

O que faz: uma conversa por solicitação, com balões (minhas mensagens à direita, do outro à esquerda) e horário. Quando a solicitação é recusada ou cancelada, o chat fica somente leitura.

No código: `ui/chat/ChatViewModel.kt` (mensagens, rascunho, "pode enviar?") e `ChatScreen.kt` (rola até a última mensagem). `ChatRepository.send` confere se quem envia é o motorista ou o passageiro daquela solicitação e se ela ainda está ativa.

Tabela: `messages` (solicitação, remetente, texto, horário). **Limite do MVP:** as mensagens ficam no banco do aparelho (para conversar entre aparelhos seria preciso uma API de chat).

7.8 Minha atividade

O que faz: quatro métricas (caronas oferecidas, passageiros confirmados, pedidos a responder, minhas solicitações) e duas abas: "Oferecidas" (com selo de pedidos pendentes) e "Solicitadas" (com botões de chat e cancelar). O ícone de lixeira limpa o histórico.

No código: `ui/activity/ActivityViewModel.kt` combina as caronas do motorista, minhas solicitações e as recebidas; calcula as métricas e o mapa de pendentes por carona. A aba escolhida usa `rememberSaveable` (estado só visual). Dois **AlertDialogs**: cancelar solicitação e limpar histórico (remove solicitações recusadas, canceladas ou de caronas que já partiram).

7.9 Contatos

O que faz: lista de amigos. Adiciona-se pelo e-mail institucional de quem já tem conta; remover pede confirmação. Caronas de contatos ganham a etiqueta "Contato" e podem ser filtradas ("Meus contatos").

No código: `data/repository/ContactRepository.kt` valida o e-mail, impede adicionar a si mesmo ou repetir, e busca o nome/vínculo da pessoa na tabela `users`. Se o motorista vem da API e não tem conta no aparelho, ele pode ser adicionado pelo detalhe da carona usando o nome da própria carona. `ui/contacts/*` traz o diálogo de adicionar (com erro no campo) e o AlertDialog de remover.

Tabela: `contacts` (dono, contato, nome, vínculo).

7.10 Perfil

- **Foto:** escolhida na galeria pelo seletor do sistema (`PickVisualMedia`, sem pedir permissão). `ProfilePhotoStorage` reduz a imagem (até ~512 px), salva como JPEG na pasta interna do app e o Coil a exibe (`UserAvatar`). Sem foto, aparecem as iniciais.
- **Editar perfil:** nome, vínculo (Estudante/Professor(a)/Colaborador(a)) e curso.

- **E-mail:** somente leitura (é a identidade da conta e a base do acesso restrito).
- **Alterar senha:** pede a senha atual, confere o hash, exige a nova diferente da atual e gera novo salt.
- **Contatos:** resumo e atalho para a aba de contatos.
- **Tema:** Sistema / Claro / Escuro, salvo no DataStore.
- **Sobre o app** e **Sair** (com confirmação).

No código: `ui/profile/ProfileViewModel.kt` (formulários e diálogos no estado), `ProfileScreen.kt` e `data/repository/ProfileRepository.kt`.

8. Banco de dados, DataStore, API e sincronização

8.1 Room (SQLite local)

Tabela	Conteúdo e relações
`users`	Conta: e-mail (chave), nome, salt, hash da senha, vínculo, curso, caminho da foto.
`rides`	Caronas: id (UUID), id remoto, motorista, saída, destino, rota, horário, vagas, veículo, observações e `synced`.
`ride_requests`	Pedidos de vaga; chave estrangeira para `rides` (apagar a carona apaga os pedidos).
`messages`	Mensagens do chat; chave estrangeira para `ride_requests`.
`contacts`	Contatos de cada usuário (chave composta dono + contato).

- **Entities** (`data/local/entity`) são as tabelas; **DAOs** (`data/local/dao`) têm as consultas: leituras retornam `Flow<List<T>>` (a tela atualiza sozinha), escritas são `suspend`.
- `CaronaDatabase` é a classe abstrata que herda de `RoomDatabase`.
- `@Upsert` grava-ou-atualiza sem apagar a linha (importante para não perder pedidos ligados à carona).

8.2 DataStore (preferências)

`UserPreferencesDataSource` guarda três coisas: o **e-mail da sessão**, o **tema** e o **último filtro** da lista (com vagas, meus contatos, período).

8.3 API REST com Retrofit

Método	Rota	Uso
GET	`/rides`	Baixar as caronas
POST	`/rides`	Enviar uma carona nova
PUT	`/rides/{id}`	Enviar uma edição
DELETE	`/rides/{id}`	Excluir no servidor

- `CaronaApi` é a interface Retrofit; `RideDto` é o JSON (kotlinx.serialization). O id do servidor aceita número ou texto (compatível com json-server 0.x e MockAPI).
- `NetworkModule` configura timeouts de 8 s e log das chamadas só no modo debug.
- `ErrorMessages.kt` traduz falhas técnicas em mensagens amigáveis ("Não foi possível conectar ao servidor...").

8.4 Sincronização offline-first

1. A tela **sempre lê do Room**; por isso o app funciona sem internet.
2. `RideRepository.refresh()`: (a) envia as caronas com `synced = false` (POST se nova, PUT se já existia); (b) baixa `GET /rides`; (c) aplica o resultado em uma transação, sem sobrescrever o que ainda está pendente de envio.
3. Cards de carona não sincronizada mostram "Aguardando sincronização com o servidor".
4. Excluir uma carona já sincronizada exige o servidor (senão ela reapareceria no próximo refresh).

9. Navegação, diálogos de confirmação e visual

9.1 Navegação

- **AuthNavHost** — login ↔ cadastro.
- **MainNavHost** com barra inferior e **quatro grafos aninhados**: Caronas (lista, detalhe, formulário), Atividade (painel, chat), Contatos e Perfil. Os destinos são rotas em `ui/navigation/Routes.kt`.
- A barra inferior só aparece nas telas principais; nas telas internas, o botão voltar e a seta da barra superior respeitam a pilha de navegação.
- Trocar de aba usa `saveState/restoreState`, então cada aba lembra onde você estava.

9.2 AlertDialog em ações críticas

Todos usam o componente `ConfirmDialog` (`ui/components/Components.kt`), com título, mensagem contextual e botões tratados **na ViewModel** (o estado do diálogo vive lá, sobrevive à rotação):

Ação	Onde
Excluir carona	Detalhe da carona
Descartar alterações do formulário	Formulário de carona (botão voltar)
Cancelar solicitação	Detalhe e aba Atividade
Recusar passageiro	Detalhe da carona (motorista)
Limpar histórico	Aba Atividade
Remover contato	Aba Contatos
Remover foto de perfil	Perfil
Sair da conta	Perfil

9.3 Visual (tema)

Em `ui/theme/Theme.kt`: paleta **verde-azulado (teal)** com acento **índigo**, tema claro e escuro, cantos arredondados (8 a 28 dp) e tipografia em semi-negrito. O app não usa HTML/CSS: o "CSS" é o `MaterialTheme`. Para mudar a identidade visual basta trocar as cores desse arquivo. Recursos de apoio em `res/`: `colors.xml` (cor do ícone), `themes.xml`, `strings.xml`, `network_security_config.xml` e o ícone adaptativo.

9.4 Estado visual com remember / rememberSaveable

Estado que só importa à aparência fica na tela com `rememberSaveable` (aba selecionada, seletor de data aberto). O resto (formulários, diálogos, dados) fica na ViewModel — é a prática de **state hoisting**.

10. Testes unitários

Arquivo	O que verifica
`EmailPolicyTest`	Domínios aceitos, falsos parecidos recusados, normalização de maiúsculas/espaços.
`RideValidationTest`	Campos obrigatórios, horário no futuro, limites de vagas.
`RideFiltersTest`	Esconde caronas que já partiram, filtro de vagas, busca sem distinção de maiúsculas, período, contatos e ordenação.
`AffiliationTest`	Texto do vínculo ("Estudante de Direito", "Professor(a)").
`PasswordHasherTest`	Mesmo salt+senha gera o mesmo hash; salt ou senha diferentes mudam o hash.

São testes de funções puras (rápidos, sem emulador): por isso a lógica de negócio foi isolada no pacote `domain`.

11. Roteiro de apresentação (8 minutos)

Primeiros 3 minutos — problema e arquitetura

1. Problema: transporte caro e carros com vagas vazias; solução: comunidade fechada por e-mail institucional.
2. Mostre o diagrama UI → ViewModel → Repository → Room/DataStore/Retrofit e cite: StateFlow, UDF, Compose, sem `Context` nas ViewModels.
3. Diga que o Room é a fonte da verdade e a API é sincronizada (offline-first).

5 minutos — demonstração ao vivo (com o backend ligado)

1. **Cadastro** com e-mail recusado (Gmail) e depois aceito (`@...edu.br`), escolhendo curso.
2. Aba **Caronas**: lista vinda da API (3 caronas de exemplo), busca e filtros.
3. **Oferecer carona**; digite algo e aperte voltar → **AlertDialog** de descarte. Depois publique.
4. Abra a carona → **Ver rota no mapa**.
5. Em **Perfil** → **Sair** (AlertDialog); crie **outro usuário**, **solicite vaga** e abra o **chat**.
6. Volte ao motorista → detalhe → **confirmar** o passageiro (vagas diminuem).
7. **Atividade**: métricas e **limpar histórico** (AlertDialog). **Contatos**: adicionar/remover. **Perfil**: foto, senha, tema.
8. Desligue o backend e toque em atualizar para mostrar o **Snackbar com "Tentar novamente"** (cache funcionando).

> **Dica:** deixe dois usuários já cadastrados e uma carona criada antes da apresentação para não perder tempo digitando.

12. Perguntas técnicas prováveis, com respostas

Pergunta	Resposta curta
Por que MVVM?	Separa tela, estado e regra de negócio; a ViewModel sobrevive à rotação e é testável. A tela só desenha o `UiState`.
Por que a ViewModel não tem `Context`?	Evita vazamento de memória e acoplamento. Quem precisa de `Context` (Room, DataStore, foto) fica na camada de dados, criado no `AppContainer`.
Qual a diferença entre `StateFlow` e `SharedFlow` aqui?	`StateFlow` guarda o estado atual da tela (sempre tem valor). `SharedFlow` envia eventos de uso único (Snackbar, voltar) que não devem repetir.
O que é UDF?	Fluxo unidirecional: estado desce (ViewModel → tela) e eventos sobem (tela → ViewModel).
O que é state hoisting?	Tirar o estado de dentro do composable e passá-lo por parâmetros; assim a tela fica "burra" e reutilizável.

Quando usar `remember` e `rememberSaveable`?	`remember` sobrevive a recomposições; `rememberSaveable` também sobrevive a rotação/morte de processo. Uso só para estado visual (aba, seletor).
Como o app funciona sem internet?	A tela lê do Room. Caronas criadas offline ficam com `synced = false` e são enviadas no próximo refresh.
Como evita passar do limite de vagas?	A confirmação roda em `withTransaction` e confere `confirmedCount < seatsTotal` antes de gravar.
Como a senha é protegida?	Nunca fica em texto: salt aleatório + SHA-256. (Em produção, o ideal é um backend com bcrypt/argon2.)
Por que `@Upsert` e não `REPLACE`?	`REPLACE` apagaria a carona e, por cascata, os pedidos ligados a ela. `@Upsert` só atualiza.
Onde ficam as corrotinas e os dispatchers?	ViewModels lançam em `viewModelScope` (Main); repositórios trocam para `Dispatchers.IO` com `withContext`.
O que acontece quando a API falha?	O erro vira mensagem amigável: tela de erro com "Tentar novamente" se não há cache; Snackbar com a mesma ação se há cache.
Por que a API é um json-server?	É um backend simples e gratuito para desenvolvimento; o contrato REST é o mesmo de um servidor real (MockAPI/Supabase).
Quais as limitações do MVP?	Autenticação local; chat e pedidos só no aparelho; exclusão de carona sincronizada exige conexão. Evoluções: backend real, chat em tempo real, mapa embutido.

13. Glossário rápido

Termo	Significado
Jetpack Compose	Toolkit moderno de interface do Android: telas são funções Kotlin (`@Composable`).
Material 3	Sistema de design do Google (componentes e temas) usado pelo Compose.
ViewModel	Classe que guarda o estado da tela e sobrevive a mudanças de configuração.
Flow / StateFlow	Fluxo de valores ao longo do tempo; `StateFlow` sempre tem o valor atual.
Corrotina	Forma leve de executar trabalho assíncrono em Kotlin (`suspend` , `launch`).
Room / DAO / Entity	Biblioteca de banco SQLite; Entity = tabela, DAO = consultas.
DataStore	Armazena pequenas preferências chave-valor (substitui SharedPreferences).
Retrofit	Cliente HTTP que transforma uma interface Kotlin em chamadas REST.
DTO	Objeto que representa o JSON da API (`RideDto`).

Repository	Camada que junta fontes de dados e esconde detalhes da UI.
sealed interface	Conjunto fechado de tipos; ideal para estados (`Loading/Success/Error`).
Offline-first	O app usa o banco local como fonte principal e sincroniza com o servidor quando possível.
Conventional Commits	Padrão de mensagens de commit (`feat:` , `fix:` , `docs:` ...).

Documento gerado a partir do código do projeto Carona Universitária. Os nomes dos integrantes e o link do repositório devem ser preenchidos no README antes da entrega.